



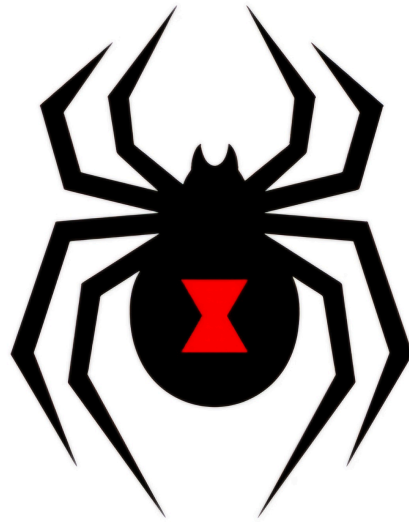
UNSA

Universidad Nacional de San Agustín

“AÑO DE LA ESPERANZA Y EL FORTALECIMIENTO DE LA DEMOCRACIA”

FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS



VIUDA NEGRA

Plan de pruebas

ASIGNATURA:

Pruebas de Software

DOCENTE:

Ing. Robert Edison Arisaca Mamani

INTEGRANTES:

Tejada Lazo Jordy Rolando (Test Lead)

Hurtado Bejarano Michael Steve (Test Analyst)

Jaita Chura Jose Manuel (Test Design)

Yare Chulunquia Kevin Pedro (Test Analyst)

Del Castillo Montoya Christopher Brad (Architect)

Índice

Índice	2
1. Introducción	4
1.1. Alcance	4
1.2. Referencias	4
1.3. Glosario	6
2. Contexto de las Pruebas	11
2.1. Proyecto / Subprocesos de Prueba	11
2.1.1 Archivos de Test de Integración — Inventario Real del Proyecto	13
2.2. Elementos de Prueba	19
2.3. Alcance de la Prueba	20
2.4. Suposiciones y Restricciones	22
2.5. Partes Interesadas	23
3. Comunicación de las Pruebas	23
4. Registro de Riesgos	26
5. Estrategia de Prueba	30
5.1. Enfoque Metodológico	30
5.2. El Pipeline CI/CD como Framework de Integración	31
5.3. Casos de Prueba Integración - Basado en repositorio Github	33
5.4. Entregables del Plan de Integración:	42
5.5. Técnicas de Diseño Aplicadas	43
5.6. Criterio de Aceptación	43
5.7. Métricas de Prueba	46
5.8. Criterios de Suspensión y Reanudación	47
5.8.1. Criterios de suspensión	47
5.8.2. Criterio de reanudación	47
6. Actividades y Estimados de Prueba	47
7. Personal	50
7.1. Roles, Actividades y Responsabilidades	50
7.2. Necesidades de Contratación	52
7.3. Necesidades de Capacitación	52
8. Cronograma	53

INVENTREE Documento de Plan de Pruebas de Integración	UNSA - EPIS Página 3
--	---------------------------------------

Control de versiones

Versión	Autor(es)	Descripción	Fecha
2.0	Tejada Lazo, Jordy Rolando	Versión inicial del documento	06/06/26
2.1	Tejada Lazo, Jordy Rolando	Se reclasifican las pruebas E2E con Playwright como pruebas de Aceptación/Sistema, excluyéndolas del alcance del Plan de Pruebas de Integración, según indicación del docente Ing. Robert Arisaca Mamani. Se actualiza el alcance, subprocesos, casos de prueba, técnicas y glosario.	Junio 2026

1. Introducción

1.1. Alcance

El presente documento constituye el Plan de Pruebas de Integración del proyecto InvenTree v1.3.0, elaborado en el contexto del curso de Pruebas de Software (UNSA-EPIS, Hito 2). Su propósito es definir el marco técnico y metodológico para planificar, organizar y eventualmente ejecutar (Hito 3) las pruebas que verifican la correcta interacción entre los componentes internos del sistema.

A diferencia de las pruebas unitarias (que aíslan cada componente), las pruebas de integración de InvenTree verifican que los módulos del backend Django se comunican correctamente con la base de datos real (PostgreSQL, MySQL), que la API REST expone los datos esperados por los clientes, que el sistema de plugins interactúa con el núcleo Django sin conflictos, y que el frontend React consume la API backend de forma completa y coherente.

El presente Plan de Pruebas de Integración así mismo define el marco para planificar, gestionar y ejecutar las pruebas de integración del sistema InvenTree v1.3.0 (open-source, licencia MIT). Su foco es exclusivamente la capa de integración entre los componentes del sistema: módulos del backend Django, la API REST, las bases de datos de producción (PostgreSQL, MySQL), el sistema de caché Redis, el sistema de plugins, y la comunicación frontend-backend validada a través del cliente Python oficial (inventree-python).

InvenTree cuenta con pruebas de integración implementadas en su pipeline CI/CD oficial (archivo `.github/workflows/qc_checks.yaml`). El objetivo de este plan es RE-EJECUTAR y documentar dichas pruebas, comprender su diseño, verificar que los componentes del sistema se integran correctamente, y sentar las bases para la ejecución que se realizará en el Hito 3.

1.2. Referencias

- Repositorio oficial de InvenTree: <https://github.com/inventree/InvenTree>
- Documentación oficial de InvenTree: <https://docs.inventree.org/>

INVENTREE Documento de Plan de Pruebas de Integración	UNSA - EPIS Página 5
--	---------------------------------------

- Cliente oficial Python (inventree-python):
<https://github.com/inventree/inventree-python>
- Repositorio de bases de datos para pruebas de migración (test-db):
<https://github.com/inventree/test-db>
- Código fuente analizado: InvenTree-master.zip (rama master)
- Workflow principal de CI/CD: [.github/workflows/qc_checks.yaml](#)
- Workflow de pruebas E2E con Playwright:
[.github/workflows/frontend.yaml](#) [REFERENCIA — clasificado como prueba de Aceptación/Sistema]
- Workflow de importación y exportación de datos:
[.github/workflows/import_export.yaml](#)
- Configuración de cobertura de código: [/codecov.yml](#)
- Plataforma Codecov (cobertura aproximada del proyecto: 87%)
- Documentación oficial de pruebas de Django:
<https://docs.djangoproject.com/en/stable/topics/testing/>
- DeepWiki – CI/CD Pipeline:
<https://deepwiki.com/inventree/InvenTree/5.1-cicd-pipeline>
- DeepWiki – Testing Strategy:
<https://deepwiki.com/inventree/InvenTree/5.2-testing-strategy>
- Archivos de pruebas de integración API analizados: [part/test_api.py](#),
[stock/test_api.py](#), [order/test_api.py](#), [build/test_api.py](#),
[company/test_api.py](#), [plugin/test_api.py](#), [users/test_api.py](#),
[common/test_api.py](#)
- Clase base de pruebas: [InvenTree/unit_test.py](#) (InvenTreeAPITestCase, InvenTreeTestCase, PluginMixin)
- Task runner del proyecto: [tasks.py](#) (dev.test, dev.setup-dev, dev.setup-test, migrate)
- ISO/IEC/IEEE 29119 – Estándar internacional para pruebas de software
- ISO/IEC/IEEE 29119-3 – Documentación de pruebas de software
- Plan de Pruebas Unitarias – InvenTree Hito 2 (documento interno del equipo)

- Plantilla “Plan de Pruebas” del curso Pruebas de Software 2025-A (UNSA – EPIS)

1.3. Glosario

En este documento se utilizan los siguientes términos abreviados:

Término	Definición
PI	Prueba de Integración: verifica la interacción entre dos o más componentes o sistemas
API REST	Interfaz de Programación basada en HTTP/REST (Django REST Framework en InvenTree)
DRF	Django REST Framework — librería para construir APIs REST en Django
ORM	Object-Relational Mapping — capa de abstracción de base de datos de Django
CI/CD	Integración y Despliegue Continuo — GitHub Actions en InvenTree
Job	Unidad de trabajo dentro de un workflow de GitHub Actions
Invoke	Herramienta Python de automatización de tareas (tasks.py en InvenTree)
Redis	Sistema de caché en memoria usado por InvenTree para sesiones y tareas async
Plugin	Componente de extensión del sistema InvenTree, cargado dinámicamente
Fixture	Datos de prueba predefinidos usados para inicializar el estado de un test
Codecov	Plataforma de medición y reporte de cobertura de código
MPTT	Modified Preorder Tree Traversal — estructura de árbol usada para categorías y ubicaciones en InvenTree

INVENTREE Documento de Plan de Pruebas de Integración	UNSA - EPIS Página 7
--	---------------------------------------

qc_checks	Nombre del workflow principal de CI/CD de InvenTree
paths-filter	Mecanismo del CI de InvenTree que ejecuta solo los jobs relevantes según archivos modificados
nyc	Herramienta de cobertura de código para JavaScript (usada en el frontend React)
inventree-python	Librería cliente oficial de InvenTree para Python; se testea contra la API real
Big-Bang	Estrategia de integración donde todos los módulos se combinan y prueban a la vez
Incremental	Estrategia de integración donde los módulos se integran y prueban uno por uno
Top-Down	Estrategia incremental que empieza desde los módulos de mayor nivel jerárquico

Término	Definición	Origen / Referencia en el proyecto
InvenTreeAPI TestCase	Clase base de todos los tests de integración API del proyecto. Hereda de APITestCase de DRF y añade fixtures, helpers de autenticación y métodos get/post/patch/delete preconfigurados	InvenTree/unit_test.py, línea 451
InvenTreeTest Case	Clase base para tests que no usan la API REST directamente (tests de	InvenTree/unit_test.py, línea 447

INVENTREE Documento de Plan de Pruebas de Integración	UNSA - EPIS Página 8
--	---------------------------------------

	modelo, lógica de negocio)	
PluginMixin	Mixin que habilita el registro de plugins en el contexto de test (activa INVENTREE_PLUGIN S_ENABLED=true)	InvenTree/unit_test.py, línea 328
fixture	Archivo YAML/JSON que pre-carga datos en la BD de prueba antes de ejecutar los tests. Ubicados en */fixtures/*.yaml	Ej: part/fixtures/part.yaml, stock/fixtures/stock.yaml
setUpTestData	Método de clase de Django que carga datos una sola vez por clase de test (más eficiente que setUp)	Django TestCase — usado en todos los test_api.py
DRF / APITestCase	Django REST Framework — librería para APIs REST; APITestCase es su clase de test con client HTTP simulado	Django REST Framework
paths-filter	Primer job del pipeline CI que detecta qué archivos cambiaron y activa/desactiva jobs descendientes	qc_checks.yaml, job paths-filter

INVENTREE Documento de Plan de Pruebas de Integración	UNSA - EPIS Página 9
--	---------------------------------------

invoke dev.test	Comando principal del task runner de InvenTree para ejecutar tests. Acepta flags --coverage, --check, --migrations, --translations	tasks.py, función test(), línea 1688
invoke dev.setup-test	Configura el entorno de pruebas: migra la BD, importa fixtures y crea datos demo. Usado por los jobs Playwright	tasks.py, función setup_test(), línea 1771
invoke int.frontend-compile	Compila el frontend React para los tests Playwright (requiere Node.js 24)	tasks.py, función frontend-compile
migration_test	Tag especial de Django que clasifica los tests de migración. Usado con --tag y --exclude-tag	tasks.py, cmd += '--exclude-tag migration_test'
CaptureQueriesContext	Utilidad de Django que captura las queries SQL generadas durante un test; usada en performance tests de la API	django.test.utils — usado en part/test_api.py, stock/test_api.py
MPTT	Modified Preorder Tree Traversal — librería usada para árboles jerárquicos de categorías y ubicaciones	Dependencia: django-mptt en requirements.txt

import_export.yaml	Workflow dedicado que prueba el ciclo completo: crear datos en PostgreSQL → exportar a JSON → importar en SQLite	github.com/inventree/InvenTree/blob/master/.github/workflows/import_export.yaml
test-db	Repositorio GitHub con BDs SQLite legacy para pruebas de migración: stable_0.10.0, 0.11.0, 0.13.5, 0.16.0, 0.17.0	https://github.com/inventree/test-db
zizmor	Herramienta de análisis de seguridad de workflows GitHub Actions — job zizmor en qc_checks.yaml	qc_checks.yaml, job zizmor
ORM	Object-Relational Mapping — capa de acceso a datos de Django que abstrae el motor de BD	Django core — todos los modelos.py

2. Contexto de las Pruebas

2.1. Proyecto / Subprocesos de Prueba

InvenTree es un sistema open-source de gestión de inventario construido sobre un backend Python/Django que expone una API REST completa. Su arquitectura multicapa hace que las pruebas de integración sean especialmente relevantes, ya que el sistema combina múltiples tecnologías y motores de base de datos en producción.

InvenTree en sí implementa una arquitectura multicapa bien definida. Cada frontera entre capas constituye un punto de integración que debe ser verificado. El

siguiente análisis fue realizado directamente sobre el código fuente del repositorio:

Repositorio GitHub	https://github.com/inventree/InvenTree (7.1k ★ · 1.4k forks · 17,657+ commits)
Versión analizada	master (rama principal, junio 2026)
Licencia	MIT License
Backend	Python 3.11 / 3.14 + Django 5.x + Django REST Framework
Frontend	React 18 + TypeScript + Mantine UI + TanStack Query + Vite
Bases de datos soportadas	SQLite (dev/test) · PostgreSQL 17 (producción) · MySQL 9 (producción)
Caché / Tareas async	Redis 8 + Django-Redis + Django Q2
CI/CD pipeline	GitHub Actions — qc_checks.yaml (691 líneas, 12+ jobs) + frontend.yaml
Tests de integración API (archivos .py encontrados)	part/test_api.py (82 métodos) · stock/test_api.py (91) · order/test_api.py (128) · build/test_api.py (48) · company/test_api.py (30) · plugin/test_api.py (25) · users/test_api.py (14) · common/test_api.py (34) = 452+ métodos de test de integración API
Fixtures de prueba verificadas	category · part · bom · stock · location · order · sales_order · transfer_order · return_order · build · company · supplier_part · users · settings
Cobertura objetivo (codecov.yml real)	Backend apps: 90% · Backend general: 92% · Frontend (web): 68% · Migrations: 40%
Cobertura mínima global (curso)	85% (el proyecto supera este umbral en el backend)

Capa	Tecnología (verificada en requirements.txt)	Rol en la integración
API Gateway / Routing	Django URL dispatcher + DRF Router	Enruta peticiones HTTP a las vistas correctas
Serialización	Django REST Framework Serializers	Convierte objetos Python ↔ JSON/XML para la API
Lógica de negocio	Django Models + Signals + Validators	Valida, procesa y persiste datos; dispara señales a plugins
Acceso a datos	Django ORM + MPTT + django-q2	Genera SQL optimizado para cada motor de BD
Persistencia	PostgreSQL 17 / MySQL 9 / SQLite	Motor de base de datos real de producción
Caché	Redis 8 + django-redis	Almacena sesiones, permisos y resultados de queries frecuentes
Tareas async	Django Q2 + Redis como broker	Procesa jobs en background (notificaciones, reportes)
Sistema de plugins	plugin/registry.py + Django Signals	Carga dinámica de módulos; hooks via pre/post_save, event signals
Cliente Python	inventree-python (repo externo)	Consume la API REST real vía HTTP; probado en job 'python'
Frontend SPA	React 18 + TanStack Query + Vite	Consume API REST; probado E2E con Playwright

2.1.1 Archivos de Test de Integración — Inventario Real del Proyecto

El siguiente inventario fue obtenido directamente del código fuente del ZIP. Los tests marcados como 'API Integration' usan InvenTreeAPITestCase y realizan peticiones HTTP reales contra la API:

Archivo de test	Clases de test	Métodos test	Fixtures usados	Tipo principal
part/test_api.py	17 clases: PartCategoryAPITest, PartAPITest, PartCreationTests, PartDetailTests, PartListTests, BomItemTest, ParameterTests, etc.	82 métodos	category, part, bom, params, location, company, stock, order, test_templates, manufacturer_ part, supplier_part	API Integration (DRF)
stock/test_api.py	14 clases: StockLocationTest, StockItemListTest, StockItemTest, StocktakeTest, StockTrackingTest, StockMergeTest, StockAssignTest, etc.	91 métodos	category, part, bom, company, location, supplier_part, stock, stock_tests, test_templates	API Integration (DRF)
order/test_api.py	15 clases: PurchaseOrderTest, PurchaseOrderReceive Test, SalesOrderTest, SalesOrderAllocateTe st, ReturnOrderTests,	128 métodos	category, part, company, location, supplier_part, stock, order,	API Integration (DRF)

	TransferOrderTest, etc.		sales_order, transfer_order	
build/test_api.py	14 clases: TestBuildAPI, BuildTest, BuildAllocationTest, BuildItemTest, BuildOverallocationTest, BuildOutputCreateTest, BuildConsumeTest, etc.	48 métodos	category, part, location, bom, build, build_line, stock	API Integration (DRF)
company/test_api.py	6 clases	30 métodos	category, part, company, location, supplier_part, manufacturer_ part, stock, order	API Integration (DRF)
plugin/test_api.py	3 clases: PluginDetailAPITest (con PluginMixin), PluginListAPITest, PluginSettingAPITest	25 métodos	— (PluginMixin configura los plugins en runtime)	API Integration + Plugin System
users/test_api.py	Clases de test de roles, tokens y permisos	14 métodos	users	API + Auth Integration
common/test_api.py	Clases de test de settings, notificaciones y estados custom	34 métodos	settings	API Integration (settings/con fig)

INVENTREE Documento de Plan de Pruebas de Integración	UNSA - EPIS Página 15
--	--

InvenTree/test_api.py	Tests del core API: versión, endpoints base	~15 métodos	—	API Integration (Core)
InvenTree/test_auth.py	TestSsoGroupSync, TestAuthViews — prueba SSO y allauth	~12 métodos	—	Auth Integration (allauth/SSO)
InvenTree/test_middleware.py	Tests del middleware de rate limiting y seguridad	~8 métodos	—	Middleware Integration
performance/tests.py	Benchmarks con pytest + inventree-python: test_api_auth_performance, URLs /api/part/, /api/stock/, etc.	~12 métodos	Servidor real corriendo	Performance Integration (cliente Python)
TOTAL	—	452+ métodos de integración API	—	—

Subprocesos de Integración (Componentes a Integrar)

Se identifican 5 subprocesos de integración, cada uno verificado por uno o más jobs del pipeline CI de InvenTree. las pruebas de integración de InvenTree cubren la interacción entre los siguientes componentes principales:

Subproceso 1: ORM Django ↔ Base de Datos Real (PostgreSQL/MySQL)

INVENTREE Documento de Plan de Pruebas de Integración	UNSA - EPIS Página 16
--	--

- Interacción entre los modelos Django (Part, Stock, Order, Build, Company, Plugin) y el motor de base de datos real (PostgreSQL / MySQL).
- Validación de constraints de integridad referencial, transacciones, índices y comportamientos específicos por motor de BD.
- Pruebas de migraciones: que los archivos migration.py se apliquen correctamente en todos los motores soportados.
- Qué verifica: que el ORM genera SQL correcto para cada motor de BD; que constraints de FK, índices, transacciones y tipos de datos funcionan.
- Job CI responsable: 'postgres' (PostgreSQL 17 + Redis 8) y 'mysql' (MySQL 9).
- Comando real: `invoke dev.test --check --translations` (ejecutado en ambos jobs)
- Variables de entorno clave — PostgreSQL:
`INVENTREE_DB_ENGINE=django.db.backends.postgresql,`
`INVENTREE_DB_USER=inventree,`
`INVENTREE_DB_PASSWORD=password,`
`INVENTREE_CACHE_HOST=localhost.`
- Variables de entorno clave — MySQL:
`INVENTREE_DB_ENGINE=django.db.backends.mysql,` imagen:
`mysql:9.`

Subproceso 2: API REST (DRF) ↔ Modelos Django ↔ SQLite + Cobertura

- Interacción entre las vistas de Django REST Framework y la capa ORM: que los endpoints retornen los datos correctos con los serializadores adecuados.
- Validación de permisos de acceso por rol de usuario en cada endpoint.
- Verificación de que los filtros, búsquedas y paginación de la API funcionan correctamente con datos reales en la BD.
- Qué verifica: que las 452+ peticiones HTTP simuladas en los `test_api.py` retornan los datos correctos vía la API; mide la cobertura de código.
- Job CI responsable: 'coverage' (matrix Python 3.11 y 3.14, SQLite en memoria).

- Flags: `INVENTREE_PLUGINS_ENABLED=true` (plugins activos durante los tests).
- Comando real: `invoke dev.test --check --coverage --translations`
- Post-ejecución: `coverage xml -i` (genera XML para Codecov) → upload con flags: backend.

Subproceso 3: Backend API ↔ Cliente Python (inventree-python)

- Que el sistema de plugins cargue, active y desactive módulos dinámicamente sin afectar el core.
- Que los hooks de señales Django (`pre_save`, `post_save`, etc.) funcionen correctamente con plugins activos.
- Pruebas ejecutadas con `INVENTREE_PLUGINS_ENABLED=true` para simular entorno de producción real.
- Qué verifica: que la API pública de InvenTree es consumible por el cliente oficial de Python.
- Job CI responsable: 'python'
- Mecanismo real (del workflow): `git clone inventree-python → invoke dev.delete-data -f → invoke dev.import-fixtures → invoke dev.server -a 127.0.0.1:12345 & → invoke wait → cd inventree-python && coverage run -m unittest discover -s test/`
- También ejecuta tests de performance con CodSpeed cuando corre en el repo oficial.

Subproceso 4: Sistema de Plugins ↔ Core Django (Señales y Hooks)

- Verificación de que la caché de permisos de usuario se almacena y expira correctamente en Redis.
- Validación de que las tareas async (Django Q2) se encolan y procesan a través de Redis.
- Qué verifica: carga, activación y desactivación de plugins; señales Django disparadas correctamente con plugins activos.
- Job CI responsable: 'coverage' y 'postgres' (ambos con `INVENTREE_PLUGINS_ENABLED=true`).
- Clase base especial: `PluginMixin` (`InvenTree/unit_test.py`, línea 328) que habilita el registry de plugins en tests.

INVENTREE Documento de Plan de Pruebas de Integración	UNSA - EPIS Página 18
--	--

- Archivos: plugin/test_api.py (25 métodos) + plugin/samples/*/test_*.py (múltiples archivos de tests de plugins sample).

Subproceso 5: Migraciones ↔ Múltiples Motores de BD

- El job 'python' en el CI levanta un servidor InvenTree real y ejecuta la librería cliente inventree-python contra él.
- Valida que la API pública de InvenTree es compatible con el cliente oficial de Python.
- Qué verifica: que los archivos migration.py se aplican sin errores en bases de datos reales; incluye upgrade desde versiones legacy.
- Job CI responsable: 'migration-tests' (PostgreSQL 17, tag --migrations) y 'migrations-checks' (SQLite, BDs legacy).
- BDs legacy usadas: stable_0.10.0.sqlite3, stable_0.11.0.sqlite3, stable_0.13.5.sqlite3, stable_0.16.0.sqlite3, stable_0.17.0.sqlite3 (del repo inventree/test-db).
- Comando real: invoke dev.test --check --migrations --report --coverage --translations

2.2. Elementos de Prueba

Se realizarán pruebas de integración sobre los siguientes elementos del sistema InvenTree:

Elemento	Tipo de componente	Archivos relevantes en el repo	Nivel de integración
part/ + stock/ + order/ + build/	Módulos Django — ORM	*/models.py, */views.py, */serializers.py	Backend ↔ BD
InvenTree/api.py + */api.py	Vistas API REST (DRF)	src/backend/InvenTree/**/api.py	API ↔ Modelos
plugin/ (sistema de plugins)	Motor de extensiones	plugin/registry.py, plugin/base.py	Plugin ↔ Core Django

Django Q2 + Redis	Tareas asíncronas + Caché	InvenTree/tasks.py, tasks.py	Backend ↔ Redis
inventree-python (librería cliente)	Cliente API oficial Python	Repositorio externo inventree-python	Cliente ↔ API REST
**/migrations/*.py	Migraciones de esquema	Todos los módulos — subcarpeta migrations/	Django ↔ BD (múltiples motores)

2.3. Alcance de la Prueba

Incluido en el alcance:

- Revisión y preparación para la re-ejecución de la suite de pruebas de integración de la API REST, compuesta por más de 452 métodos de prueba distribuidos en los archivos test_api.py de los distintos módulos.
- Verificación de la configuración y cobertura del pipeline de Integración Continua (CI), incluyendo los flujos qc_checks.yaml, frontend.yaml e import_export.yaml.
- Pruebas de integración entre los módulos backend desarrollados en Django y las bases de datos soportadas:
 - PostgreSQL 17 (postgres)
 - MySQL 9 (mysql)
 - SQLite, incluyendo medición de cobertura de código (coverage)
- Validación de la integración con Redis 8 como sistema de caché y broker de mensajes.
- Verificación de la integración entre la API REST y el cliente oficial de Python inventree-python (python).
- Validación del sistema de plugins y extensiones, incluyendo señales (signals), hooks y mecanismos de integración con el núcleo de Django.

INVENTREE Documento de Plan de Pruebas de Integración	UNSA - EPIS Página 20
--	--

- Pruebas de migración y actualización de base de datos, incluyendo escenarios de upgrade desde las versiones 0.10.0 hasta 0.17.0 (migration-tests, migrations-checks).
- Validación del ciclo completo de importación y exportación de datos entre PostgreSQL y SQLite.
- Comparación y validación del esquema OpenAPI generado frente al esquema de referencia previo (schema).

Excluido del alcance en este Hito (Hito 2):

- Ejecución efectiva de las pruebas de integración y automatizadas (programada para el Hito 3). Las pruebas E2E con Playwright quedan **EXCLUIDAS** de este plan al ser clasificadas como Pruebas de Aceptación/Sistema.
- Pruebas de rendimiento, carga y estrés bajo escenarios de concurrencia.
- Pruebas de seguridad avanzadas, auditorías de vulnerabilidades y pruebas de penetración (Penetration Testing).
- Integración y validación con servicios externos reales de producción, tales como servidores SMTP y proveedores OAuth.
- Pruebas de la aplicación móvil complementaria (companion app) de InvenTree (NO INCLUIDO).
- Pruebas relacionadas con APIs, dispositivos o servicios de terceros, incluyendo lectores de códigos de barras, impresoras de etiquetas y plugins externos específicos.

2.4. Suposiciones y Restricciones

Suposiciones:

- El equipo ha hecho fork del repositorio en GitHub y habilitado GitHub Actions en el fork.
- Al menos un miembro tiene Docker instalado para levantar PostgreSQL 17 y Redis 8 localmente.
- Python 3.11 y Node.js 24 están disponibles o serán instalados por el task runner.
- Se usará la demo pública <https://demo.inventree.org> (admin/inventree) como entorno de validación manual si fuera necesario.

Restricciones:

- El equipo no tiene acceso a un entorno de producción real; las pruebas de integración se ejecutan en entorno local o en el CI del fork de GitHub.
- La corrección de defectos en el código fuente de InvenTree queda fuera del alcance del curso (es un proyecto de terceros).
- El código fuente de InvenTree no será modificado (es un proyecto de terceros); solo se re-ejecutan sus tests existentes.
- Los runners de GitHub Actions gratuitos tienen limitación de 2,000 minutos/mes para repos privados. Se usará el fork público para evitar esta restricción.
- Los jobs 'migration-tests' y 'migrations-checks' solo se activan cuando hay cambios en archivos de migración o con el label 'full-run'. En el fork del equipo pueden requerir forzar la ejecución.
- Las pruebas de integración con Redis requieren que el servicio Redis esté corriendo (disponible en Docker).

2.5. Partes Interesadas

Rol	Responsabilidades
Instructor/Docente a cargo	Aprobación de criterios de aceptación académicos, realización de UAT, validación y aprobación del plan de prueba, definición de escenarios de uso real.
Test Lead	Liderazgo y coordinación del equipo de pruebas, planificación y supervisión de actividades de testing, comunicación con stakeholders, gestión de riesgos y escalaciones, aprobación de entregables de prueba.
Test Analyst	Análisis de requisitos para pruebas, diseño y ejecución de casos de prueba, reporte y seguimiento de defectos, pruebas manuales y exploratorias, validación de criterios de aceptación.
Test Architect	Definición de la arquitectura de pruebas, diseño de estrategias de testing, selección de herramientas y frameworks, establecimiento de estándares y mejores prácticas, supervisión técnica del proceso.
Test Design	Diseño detallado de casos de prueba, creación de datos de

	prueba, desarrollo de scripts de automatización, diseño de ambientes de testing, documentación técnica de pruebas.
--	--

3. Comunicación de las Pruebas

En esta sección se explican las pautas para comunicar de manera efectiva durante el proceso de pruebas.

Se define cómo debe ser la comunicación dentro del equipo y con las personas o grupos externos que estén involucrados. Además, se especifica quién es responsable de comunicar qué, por qué medios se debe hacer, con qué frecuencia y qué hacer cuando surjan conflictos o desacuerdos.

Objetivo

Asegurar que todos los miembros del equipo estén informados sobre el avance de las pruebas, defectos encontrados, prioridades de resolución, y mejoras del proceso.

Garantizar que todos los miembros del equipo conozcan el estado del pipeline CI/CD en el fork, los defectos encontrados durante la configuración del entorno y los resultados de ejecución de los jobs de integración.

Protocolo de Comunicación

- Comunicación Interna: Se usará la red social (WhatsApp) para conversaciones rápidas, reuniones sincrónicas por Google Meet, y documentos colaborativos (Google Docs / Hoja de cálculo / Plataforma Clik Cup) para registro de acuerdos.
- Comunicación Externa: Se notificará al docente mediante los servicios de correo institucional o plataforma virtual de aprendizaje.
- Resolución de Conflictos: Se gestionará en primera instancia por el Test Lead. Si no se resuelve, se eleva al Docente.

Punto de Comunicación	Propósito	Frecuencia	Canal	Responsable	Audiencia
-----------------------	-----------	------------	-------	-------------	-----------

INVENTREE Documento de Plan de Pruebas de Integración	UNSA - EPIS Página 23
--	--

Kick-off de integración	Explicar el pipeline CI/CD de InvenTree; asignar roles por job	Una vez (inicio Sprint 2)	Google Meet + Pizarra	Test Lead	Todo el equipo
Stand-up diario	Reporte: job ejecutado / bloqueado / resultado	Diario	WhatsApp group	Test Lead	Todo el equipo
Notificación de GitHub Actions	Alerta automática de job fallido en el fork	Cada push/PR al fork	GitHub Email + Actions tab	GitHub (automático)	Test Architect + Test Lead
Registro de defecto de entorno	Documentar fallo de configuración (no fallo del código)	Al encontrar un problema	GitHub Issues (label: entorno)	Test Analyst / Architect	Test Lead
Reporte semanal de avance	Resumen de jobs ejecutados, PASS/FAIL, cobertura actual	Semanal	Drive + Meet	Test Lead	Todo el equipo + Docente
Actualización del Wiki	Publicar resultados de jobs en la página de integración del Wiki	Tras cada ejecución relevante	GitHub Wiki del fork	Test Design	Docente + Equipo

Presentación Hito 2	Entrega del Plan de Integración (este documento)	10 Jun 2026	Presencial/Meet	Test Lead + Todos	Docente
Presentación Hito 3	Informe de resultados de integración + demo del CI funcionando	Fecha Hito 3	Presencial/Meet	Test Lead + Architect	Docente

Participantes del equipo:

1. Robert Edison Arisaca Mamani (Docente del curso)
2. Tejada Lazo, Jordy Rolando (Test Lead)
3. Hurtado Bejarano, Michael Steve (Test Analyst)
4. Yare Chullunquia, Kevin Pedro (Test Analyst)
5. Del Castillo Montoya, Brad Christopher (Test Architect)
6. Tejada Lazo, Jordy Rolando (Test Design)
7. Jaita Chura, Jose Manuel (Test Design)

Comentarios:

Todos los acuerdos relevantes se documentan en la carpeta compartida del equipo “Purebas_ViudaNegra” en el Drive. Se fomenta la comunicación asertiva y la colaboración proactiva.

4. Registro de Riesgos

En la siguiente tabla se identifican los riesgos del proyecto, así como se determina la severidad de cada uno de los riesgos multiplicando el impacto por la probabilidad de ocurrencia.

El impacto y la probabilidad se determinan teniendo en cuenta una escala de 1 al 5, donde 5 es el más alto.

Nº	Riesgo identificado	Prob. (1-5)	Impacto (1-5)	Severidad	Plan de Mitigación
1	Dificultad para configurar PostgreSQL + Redis localmente (Docker)	3	4	12	Usar la demo pública (demo.inventree.org) como alternativa; documentar configuración Docker paso a paso en el Wiki.
2	GitHub Actions del fork no se ejecutan por falta de secrets (CODECOV_TOKEN, etc.)	4	3	12	Ejecutar pruebas localmente con 'invoke dev.test'; los jobs de CI se pueden verificar sin el token de Codecov usando la variable de entorno vacía.
3	Tiempo de ejecución del pipeline CI supera los recursos del runner gratuito de GitHub	3	3	9	Ejecutar solo los jobs críticos (coverage, postgres) usando el label 'full-run' selectivamente; documentar tiempos en el Wiki.
4	Diferencias entre la BD SQLite (entorno local) y PostgreSQL (CI) causan fallos inesperados	3	4	12	Priorizar la ejecución con PostgreSQL vía Docker; documentar las diferencias de comportamiento encontradas.
5	El sistema de plugins genera efectos secundarios en los tests de integración	2	3	6	Asegurarse de usar INVENTREE_PLUGINS_ENABLED=true igual que en el CI oficial; no modificar plugins del repo.

INVENTREE Documento de Plan de Pruebas de Integración	UNSA - EPIS Página 26
--	--

6	Versión de Python o Django incompatible en el entorno local del equipo	3	4	12	Usar Docker o el devcontainer incluido en el repositorio (.devcontainer/) para garantizar el entorno correcto.
7	Fallos en migraciones al probar upgrade desde versiones legacy (v0.10.0)	2	4	8	Estas pruebas se delegan al job 'migration-tests' del CI; el equipo solo documenta el resultado, no las ejecuta localmente.
9	Retraso de algún integrante en la configuración del entorno	3	3	9	Asignar al Test Architect la responsabilidad de crear una guía de setup documentada en el Wiki antes de la primera sesión de pruebas.
10	Los tests de integración con inventree-python requieren un servidor en ejecución	3	3	9	El job 'python' del CI gestiona esto automáticamente; el equipo solo necesita verificar que el job pasa en GitHub Actions.

Riesgos Técnicos

ID	Riesgo	P	I	Sev.	Mitigación / Contingencia
RT-01	El job 'postgres' falla en el fork por problema de conexión a la BD (INVENTREE_DB_HOST mal configurado, puerto no expuesto en el runner)	3	4	12	Verificar que las variables de entorno del fork coinciden exactamente con qc_checks.yaml; usar el template de .env del proyecto (contrib/env.sample)

INVENTREE Documento de Plan de Pruebas de Integración	UNSA - EPIS Página 27
--	--

RT-02	El job 'coverage' falla con error de compilación de traducciones (compilemessages requiere gettext instalado)	3	3	9	Los jobs de CI incluyen apt-dependency: gettext en su setup action. Localmente: apt-get install gettext antes de ejecutar invoke dev.test --translations
RT-03	GitHub Actions del fork no ejecuta jobs por falta del secret CODECOV_TOKEN	4	2	8	El upload a Codecov fallará, pero los tests sí corren. Los jobs tienen continue-on-error para el upload de cobertura. El equipo puede verificar cobertura localmente con coverage report
RT-04	El job 'migration-tests' no se activa (requiere cambios en archivos de migración o label 'full-run')	4	3	12	Añadir el label 'full-run' a una PR en el fork para forzar la ejecución completa. O usar el workflow import_export.yaml que siempre corre el ciclo PG → SQLite
RT-05	El job 'chromium' de Playwright (4 shards) falla por timeout o recursos insuficientes en el runner gratuito	4	3	12	Ejecutar con --shard=1/4 solamente en primera instancia. El job 'firefox' (2 shards) es menos costoso. Ambos tienen timeout-minutes: 60
RT-06	Diferencias de comportamiento entre SQLite (local) y PostgreSQL (CI) causan que algunos tests pasen en uno y fallen en otro	3	4	12	El proyecto ya está diseñado para soportar los 3 motores; la mayoría de diferencias ya están corregidas. Documentar cualquier discrepancia encontrada.
RT-07	El task runner (invoke) no está disponible en el entorno local del equipo	4	3	12	pip install invoke (es una dependencia única). Todo el setup se hace vía invoke dev.setup-dev.

					Alternativamente, usar el devcontainer del proyecto (.devcontainer/) con Docker.
RT-08	Node.js versión incorrecta para los jobs del frontend (requiere v24, según env.node_version en los workflows)	3	3	9	Instalar nvm y usar nvm use 24. El setup action del proyecto gestiona esto automáticamente en CI.
RT-09	El job 'python' requiere que el servidor InvenTree esté corriendo en background; puede haber conflictos de puertos	3	3	9	El workflow usa el puerto 12345 (no el 8000 por defecto) para evitar conflictos. Localmente verificar que el puerto esté libre.
RT-10	Retraso en la ejecución de tests por parte de algún integrante del equipo	3	3	9	Asignación clara de jobs en la matriz RACI. El Test Architect ejecuta los jobs críticos (coverage + postgres) como mínimo para tener resultados de referencia.

5. Estrategia de Prueba

5.1. Enfoque Metodológico

El enfoque adoptado es el de Integración Incremental Top-Down guiada por el pipeline CI/CD oficial del proyecto. Esta decisión está fundamentada en la arquitectura del sistema y en la estructura real de los workflows del repositorio, que fueron analizados directamente:

- Top-Down: Los tests de la API (capa de presentación) se ejecutan primero con la BD más simple (SQLite), y luego se replica con los motores de producción (PostgreSQL, MySQL).
- Incremental: Los jobs del CI tienen dependencias explícitas (needs:) que garantizan un orden: code-style → coverage/postgres/mysql → python → frontend.
- CI-Driven: Los tests de integración no se ejecutan ad-hoc sino como parte del pipeline automatizado, garantizando reproducibilidad y consistencia.
- Re-use del Framework Existente: El equipo NO escribe nuevos tests de integración. Re-ejecuta los 452+ métodos existentes en los test_api.py del proyecto y documenta los resultados.

5.2. El Pipeline CI/CD como Framework de Integración

El archivo `.github/workflows/qc_checks.yaml` (691 líneas, analizado completamente) define el siguiente pipeline. Cada job corresponde a un nivel de integración:

Job (nombre en workflow)	Imagen de BD / Servicios	Trigger (paths-filter)	Qué integra (verificado en el código)	Tiempo aprox. CI
paths-filter	—	Siempre (todos los pushes)	Detecta cambios en server, migrations, frontend, api, cicd, requirements	< 1 min
code-style (Style [prek])	—	cicd, server, frontend, requirements	Ruff (linting Python), check_version.py, pre-commit hooks (gitleaks, etc.)	~2 min
typecheck (Style [Typecheck])	—	server, requirements	Verificación de tipos Python con 'ty check' — nueva herramienta Astral	~3 min
schema (Tests - API Schema)	SQLite	server, api	Genera schema OpenAPI con invoke dev.schema, lo	~3 min

INVENTREE Documento de Plan de Pruebas de Integración	UNSA - EPIS Página 30
--	--

			compara con versión previa; detecta breaking changes en la API	
coverage (Tests - DB [SQLite])	SQLite (matrix: Py3.11 + Py3.14)	server	ORM ↔ SQLite ↔ API REST (452+ métodos) ↔ Plugins (PLUGINS_ENABLED=true) — genera XML de cobertura para Codecov (flag: backend)	~8-10 min (×2 por matrix)
postgres (Tests - DB [PostgreSQL])	postgres:17 + redis:8	server	ORM ↔ PostgreSQL 17 ↔ Redis 8 ↔ API REST ↔ Plugins ↔ Data Export/Import (action: migration)	~10 min
mysql (Tests - DB [MySQL])	mysql:9	server	ORM ↔ MySQL 9 ↔ API REST ↔ Plugins — compatibilidad charset/collation/tipos MySQL	~9 min
python (Tests - inventree-python)	SQLite (servidor real en bg)	server, api	Cliente inventree-python ↔ API REST real (HTTP en 127.0.0.1:12345) ↔ Fixtures cargadas	~8 min
migration-tests (Tests - Migrations)	postgres:17	migrations (o full-run label)	Migraciones ↔ PostgreSQL: invoke dev.test --tag migration_test, cobertura flag: migrations	~12 min
migrations-checks (Tests - Full Migration SQLite)	SQLite, BDs legacy	migrations (o full-run label)	Upgrade desde 0.10.0, 0.11.0, 0.13.5, 0.16.0, 0.17.0 → invoke migrate (test-db repo)	~8 min
zizmor (Security [Zizmor])	—	cicd	Análisis seguridad de los workflows .github/workflows/*.yaml — output SARIF	~2 min

[frontend.yaml] build	—	Siempre (independiente)	Build producción del frontend React: yarn compile + yarn lib + yarn build	~5 min
[import_export.yaml] test	postgres:17	server (cambios backend)	Ciclo completo: crear datos en PG con plugins → export JSON → import en SQLite → verificar integridad	~12 min

5.3. Casos de Prueba Integración - Basado en repositorio Github

Los siguientes casos de prueba han sido extraídos directamente de los archivos test_api.py del proyecto. Se presentan los más representativos por módulo, indicando la clase y método exactos para que el equipo pueda localizarlos en el código.

Integración API ↔ Módulo Part ↔ BD (part/test_api.py — 82 métodos)

ID	Clase real en test_api.py	Método / Escenario	URL de la API	Fixtures requeridos	Resultado esperado	Job CI
INT-001	PartCategoryAPITest	test_category_list — lista categorías con filtros cascade/parent	GET /api/part/category/	category, part, bom, company, stock	HTTP 200; ({}, 8 categorías), ({parent:1,cascade:False}, 3), ({parent:1,cascade:True}, 5)	coverage
INT-002	PartAPITest	test_part_list — lista partes con filtros (cascade, search, category)	GET /api/part/	category, part, bom, company, stock	HTTP 200; lista paginada con resultados según filtros	coverage
IN	PartCreationTests	test_create_part — crea una	POST /api/part/	category	HTTP 201; pk asignado, datos	coverage

T-003		parte vía POST con datos válidos			persistidos en BD	
INT-004	PartAPITest	test_part_permissions — acceso sin auth y con roles insuficientes	POST/DELETE /api/part/	—	HTTP 401 sin token; HTTP 403 sin permisos de escritura	coverage
INT-005	BomItemTest	test_bom_create / test_bom_list — CRUD de líneas de BOM	POST/GET /api/bom/	category, part, bom	HTTP 201/200; BomItems creados y listados correctamente	coverage
INT-006	ParameterTests	test_part_parameter_api — CRUD de parámetros con unidades Pint	GET/POST /api/part/parameter/	parameters	HTTP 200/201; parámetros con unidades válidas aceptados, inválidos rechazados	coverage
INT-007	PartAPIAggregationTest	test_part_count / test_stock_aggregation — campos calculados en respuesta API	GET /api/part/{pk}/	category, part, stock	HTTP 200; campos 'in_stock', 'unallocated_stock' calculados correctamente	coverage/postgres

Integración API ↔ Módulo Stock ↔ BD (stock/test_api.py — 91 métodos)

ID	Clase real en test_api.py	Método / Escenario	URL de la API	Fixtures requeridos	Resultado esperado	Job CI
----	---------------------------	--------------------	---------------	---------------------	--------------------	--------

INVENTREE Documento de Plan de Pruebas de Integración	UNSA - EPIS Página 33
--	--

IN T- 00 8	StockLocationTest	test_list — lista ubicaciones con filtros parent/cascade/depth	GET /api/stock/location/	location , stock	HTTP 200; ({parent:1,cascade:False}, 2), ({cascade:True,depth:0}, 3)	coverage
IN T- 00 9	StockItemListTest	test_filter_by_part — filtra stock por parte específica	GET /api/stock/?part={pk}	category, part, location , stock	HTTP 200; solo los StockItems de la parte solicitada	coverage
IN T- 01 0	StockItemTest	test_stock_item_create / test_add / test_remove	POST /api/stock/ + /api/stock/{pk}/add/	part, location , stock	HTTP 201 al crear; HTTP 200 al agregar; cantidad actualizada en BD; StockItemTracking creado	postgres
IN T- 01 1	StocktakeTest	test_stocktake — ajuste de inventario (count)	POST /api/stock/{pk}/count/	stock, location	HTTP 200; cantidad = valor del conteo en la BD; tracking de ajuste registrado	postgres
IN T- 01 2	StockTrackingTest	test_stock_tracking — historial de movimientos de un ítem	GET /api/stock/tracking/?item={pk}	stock, location	HTTP 200; eventos en orden cronológico; tipo de evento correcto (add, remove, transfer)	coverage
IN T- 01 3	StockMergeTest	test_merge — combinar dos StockItems del mismo Part	POST /api/stock/merge/	stock, location , part	HTTP 200; un único StockItem resultante con cantidad sumada	postgres
IN T- 01 4	StockAssignTest	test_assign_to_customer — asignar stock a cliente	POST /api/stock/assign/	stock, company	HTTP 200; campo customer actualizado en StockItem	coverage

Integración API ↔ Módulo Order ↔ BD (order/test_api.py — 128 métodos)

ID	Clase real en test_api.py	Método / Escenario	URL de la API	Fixtures requeridos	Resultado esperado
IN-T-015	PurchaseOrderTest	test_create_purchase_order — crear PO con proveedor y referencia	POST /api/order/po/	company, supplier_part, stock, order	HTTP 201; PO creada con status 'Pending'; proveedor asignado correctamente
IN-T-016	PurchaseOrderReceiveTest	test_receive — recepción parcial y total de líneas de PO	POST /api/order/po/{pk}/receive/	company, stock, order, supplier_part	HTTP 200; StockItems creados tras recepción; cantidad recibida registrada; status PO actualizado
IN-T-017	SalesOrderTest	test_create_sales_order — crear SO con cliente	POST /api/order/so/	company, stock, order, sales_order	HTTP 201; SO creada con status 'Pending'; cliente asignado
IN-T-018	SalesOrderAllocateTest	test_allocate — asignar stock a líneas de SO	POST /api/order/so/{pk}/allocate/	company, stock, order, sales_order	HTTP 200; SalesOrderAllocation creada; stock_allocated actualizado en Part
IN-T-019	ReturnOrderTests	test_return_order_flow — ciclo completo de devolución	POST /api/order/ro/ + estado transitions	company, stock	HTTP 201/200; ReturnOrder con estados draft→in_progress→complete; stock devuelto
IN-T-	TransferOrderTest	test_transfer_order — transferencia de	POST /api/order/to/	stock, location,	HTTP 201/200; TransferOrder creada y

INVENTREE Documento de Plan de Pruebas de Integración	UNSA - EPIS Página 35
--	--

0 2 0		stock entre ubicaciones		transfer_order	ejecutada; stock en ubicación destino confirmado
-------------	--	-------------------------	--	----------------	--

Integración API ↔ Módulo Build/BOM ↔ BD (build/test_api.py — 48 métodos)

ID	Clase real en test_api.py	Método / Escenario	URL de la API	Fixtures requeridos	Resultado esperado	Job CI
INT-021	TestBuildAPI	test_get_build_list — lista de BuildOrders con filtros (status, part, active)	GET /api/build/	category, part, location, bom, build, build_line, stock	HTTP 200; 5 builds totales; 4 completadas (status:COMPLETE); 1 activa	coverage
INT-022	BuildAllocationTest	test_allocate — asignar stock al BOM de un Build	POST /api/build/{pk}/auto-allocate/	part, location, bom, build, stock	HTTP 200; BuildItem creados por cada línea del BOM; stock_allocated actualizado	postgres
INT-023	BuildOutputCreateTest	test_create_build_output — crear salidas del proceso de construcción	POST /api/build/{pk}/create-output/	part, build, bom, stock	HTTP 201; StockItems de salida creados con la cantidad especificada	postgres
INT-024	BuildConsumeTest	test_consume_build_items — consumir materiales del	POST /api/build/{pk}/finish/	part, build, bom, build_line	HTTP 200; stock de componentes decrementado; build	postgres

INVENTREE Documento de Plan de Pruebas de Integración	UNSA - EPIS Página 36
--	--

		BOM al completar Build		ne, stock	marcado como COMPLETE	
--	--	---------------------------	--	--------------	-----------------------------	--

Integración API ↔ Sistema de Plugins (plugin/test_api.py — 25 métodos)

ID	Clase real en test_api.py	Método / Escenario	URL de la API	Precondición	Resultado esperado	Job CI
INT-025	PluginDetailAPI Test (con PluginMixin)	test_plugin_uninstall — intentar desinstalar plugins con restricciones	PATCH /api/plugin/{plugin}/uninstall/	Superuser; plugins 'sample', 'bom-exporter', 'inventree-slack-notification'	HTTP 400 para plugins sample/builtin/active con mensajes específicos : 'cannot be uninstalled as it is a sample plugin'	coverage
INT-026	PluginDetailAPI Test	test_plugin_activate/deactivate — activar y desactivar plugins vía API	PATCH /api/plugin/{pk}/	Plugin existente en PluginConfig	HTTP 200; campo 'active' cambia correctamente; hooks de señales Django activados/desactivados	coverage/postgres

INT-027	PluginListAPItest	test_plugin_list — listar todos los plugins registrados	GET /api/plugin/	INVENTREE_PLUGINS_ENABLED=true	HTTP 200; lista de plugins con sus metadatos (nombre, versión, active, builtin)	coverage
---------	-------------------	---	------------------	--------------------------------	---	----------

Integración Migraciones ↔ Motores de BD

ID	Job CI (workflow)	Escenario	Motor de BD	Versiones legacy	Resultado esperado
INT-028	migrations-checks (qc_checks.yaml)	Upgrade desde BD v0.10.0: cp test-db/stable_0.10.0.sqlite3 → invoke migrate	SQLite	0.10.0	invoke migrate ejecuta sin errores; todas las migraciones aplicadas; BD íntegra post-migración
INT-029	migrations-checks	Upgrade desde v0.11.0	SQLite	0.11.0	Igual que INT-028
INT-030	migrations-checks	Upgrade desde v0.13.5	SQLite	0.13.5	Igual que INT-028
INT-031	migrations-checks	Upgrade desde v0.16.0 y v0.17.0	SQLite	0.16.0, 0.17.0	Igual que INT-028
INT-032	migration-tests (qc_checks.yaml)	invoke dev.test --tag migration_test en PostgreSQL 17	PostgreSQL 17	BD actual	Tests de migración con tag migration_test pasan; cobertura enviada a Codecov (flag: migrations)

INVENTREE Documento de Plan de Pruebas de Integración	UNSA - EPIS Página 38
--	--

IN T- 03 3	postgres (qc_checks.y aml)	Data Export/Import via action migration	Postgre SQL 17 → SQLite	—	invoke dev.export-records genera JSON válido; invoke dev.import-records restaura los mismos datos
IN T- 03 4	import_expo rt.yaml	Ciclo completo PG con plugins → JSON → SQLite	Postgre SQL 17 → SQLite	—	Exportación con plugins habilitados (INVENTREE_PLUGINS_MAN DATORY=dummy-app-plugin) → importación exitosa en SQLite

Integración API ↔ Cliente Python (inventree-python)

ID	Job CI	Mecanismo real (del workflow)	Escenario	Resultado esperado
INT-0 38	python (qc_checks.y aml)	git clone inventree-python → invoke dev.delete-data -f → invoke dev.import-fixtures → invoke dev.server -a 127.0.0.1:12345 &	Autenticación del cliente Python contra la API real	InvenTreeAPI(url, username, password) conecta y retorna token válido; coverage run ejecuta discover
INT-0 39	python	Servidor InvenTree corriendo con fixtures cargadas	Cliente lista partes, stock, empresas vía API	Part.list(), StockItem.list(), Company.list() retornan objetos Python con atributos correctos
INT-0 40	python	Servidor con datos demo	Performance benchmarks (CodSpeed) — solo en repo oficial inventree/InvenTre e	pytest ./src/performance --codspeed mide tiempos de response de las URLs de API críticas

5.4. Entregables del Plan de Integración:

Entregable	Descripción	Hito	Responsable
Plan de Pruebas de Integración v2.0 (este documento)	Define el marco técnico completo basado en el código fuente real del proyecto	Hito 2	Test Lead + Test Architect
Capturas del pipeline CI funcionando en el fork	Screenshots de cada job en la pestaña Actions del fork del equipo	Hito 3	Test Architect
Informe de Resultados de Pruebas de Integración	Tabla de resultados por job: PASS/FAIL, tiempo, cobertura obtenida, defectos	Hito 3	Test Analyst + Test Design
GitHub Issues de defectos de integración	Issues etiquetados 'prueba-integracion' en el fork para cada fallo encontrado	Hito 3	Test Analyst
Actualización del GitHub Wiki — Página Plan-Integracion	Publicación de este documento (o resumen) en el Wiki del fork	Hito 2	Test Design
Actualización del GitHub Wiki — Página Informe-Integracion	Resultados reales de ejecución del CI, métricas y evidencias	Hito 3	Test Design
Reporte de Codecov del fork	Badge y reporte de cobertura tras ejecución del job 'coverage'	Hito 3	Test Architect

5.5. Técnicas de Diseño Aplicadas

- Diseño basado en contrato de API (Contract Testing): los tests de integración verifican que cada endpoint retorna exactamente el formato y datos que el esquema OpenAPI define. El job 'schema' valida esto automáticamente.
- Pruebas dirigidas por datos (Data-Driven): cada clase de test usa fixtures específicas (declaradas en el atributo 'fixtures' de cada clase) para garantizar un estado inicial reproducible y conocido.
- Pruebas de comportamiento por motor de BD (Cross-Database Compatibility): los mismos tests se ejecutan contra 3 motores distintos para detectar diferencias de comportamiento SQL.
- Pruebas de regresión de migraciones (Migration Regression): el uso del repositorio test-db con BDs de 5 versiones históricas garantiza que ninguna actualización rompa bases de datos existentes.

5.6. Criterio de Aceptación

Criterio	Condición de éxito	Prioridad	Verificado en
Todos los jobs críticos de CI pasan	0 fallos en: coverage (SQLite), postgres (PG17+Redis8), mysql (MySQL9)	ALTA	GitHub Actions del fork
Cobertura backend $\geq$ 85%	Codecov reporta $\geq$ 85% en flag 'backend' tras job 'coverage'	ALTA (requisito del curso)	Codecov + badge en README del fork
Cobertura backend-apps $\geq$ 90%	Codecov reporta $\geq$ 90% en el componente 'Backend Apps' (target real en codecov.yml)	MEDIA	Codecov component view

INVENTREE Documento de Plan de Pruebas de Integración	UNSA - EPIS Página 41
--	--

Cobertura frontend $\geq$ 68%	Codecov reporta $\geq$ 68% en flag 'web' tras jobs chromium + merge (target real en codecov.yml)	MEDIA	Codecov flag 'web'
API schema sin breaking changes	Job 'schema' pasa sin diferencias entre schema generado y schema publicado	ALTA	GitHub Actions — job schema
Migraciones sin errores	Jobs migration-tests y migrations-checks pasan (si se activan con full-run)	ALTA	GitHub Actions
Ciclo import/export funcional	import_export.yaml pasa: datos creados en PG $\rightarrow$ exportados $\rightarrow$ importados en SQLite sin pérdida	ALTA	GitHub Actions
Cliente Python compatible	Job 'python' pasa: inventree-python se conecta y ejecuta operaciones CRUD contra la API real	MEDIA	GitHub Actions — job python
Todos los jobs críticos de CI pasan	0 fallos en: coverage (SQLite), postgres (PG17+Redis8), mysql (MySQL9)	ALTA	GitHub Actions del fork

INVENTREE Documento de Plan de Pruebas de Integración	UNSA - EPIS Página 42
--	--

Cobertura backend $\geq$ 85%	Codecov reporta $\geq$ 85% en flag 'backend' tras job 'coverage'	ALTA (requisito del curso)	Codecov + badge en README del fork
Cobertura backend-apps $\geq$ 90%	Codecov reporta $\geq$ 90% en el componente 'Backend Apps' (target real en codecov.yml)	MEDIA	Codecov component view
Cobertura frontend $\geq$ 68%	Codecov reporta $\geq$ 68% en flag 'web' tras jobs chromium + merge (target real en codecov.yml)	MEDIA	Codecov flag 'web'
API schema sin breaking changes	Job 'schema' pasa sin diferencias entre schema generado y schema publicado	ALTA	GitHub Actions — job schema
Migraciones sin errores	Jobs migration-tests y migrations-checks pasan (si se activan con full-run)	ALTA	GitHub Actions
Ciclo import/export funcional	import_export.yaml pasa: datos creados en PG $\rightarrow$ exportados $\rightarrow$ importados en SQLite sin pérdida	ALTA	GitHub Actions
Cliente Python compatible	Job 'python' pasa: inventree-python se conecta y ejecuta	MEDIA	GitHub Actions — job python

INVENTREE Documento de Plan de Pruebas de Integración	UNSA - EPIS Página 43
--	--

	operaciones CRUD contra la API real		
--	--	--	--

5.7. Métricas de Prueba

Métrica	Cómo medirla	Valor objetivo
Tasa de éxito del pipeline CI	$(\text{Jobs PASS} / \text{Total jobs ejecutados}) \times 100\%$	100% para los jobs críticos
Cobertura de código backend	coverage report (local) o Codecov (CI), flag: backend	$\geq 85\%$ (curso) / $\geq 90\%$ (codecov.yml)
Cobertura frontend	npx nyc report $\rightarrow$ lcov $\rightarrow$ Codecov, flag: web	$\geq 68\%$ (target real codecov.yml)
Casos de integración INT-001 a INT-040 ejecutados	Revisión del log de cada job de CI en GitHub Actions	100% en el Hito 3
Defectos de integración encontrados	GitHub Issues con label 'prueba-integracion'	0 (esperado, dado que re-ejecutamos tests existentes del proyecto)
Tiempo total del pipeline (paralelo)	Tiempo desde paths-filter hasta el último job (en GitHub Actions)	< 60 minutos
Versiones legacy de BD testeadas	Conteo de BDs de test-db procesadas por migrations-checks	5 (0.10.0, 0.11.0, 0.13.5, 0.16.0, 0.17.0)

5.8. Criterios de Suspensión y Reanudación

5.8.1. Criterios de suspensión

Las pruebas serán suspendidas temporalmente si se presenta alguna de las siguientes condiciones:

- Más de 3 jobs críticos fallan simultáneamente por problemas no relacionados con el código (configuración de entorno, secrets, permisos).
- El runner de GitHub Actions del fork está inaccesible o tiene problemas persistentes.
- No se puede instanciar la BD de prueba (invoke migrate falla repetidamente con errores de conectividad).

5.8.2. Criterio de reanudación

Las pruebas se reanudarán cuando:

- El Test Architect resuelve los problemas de configuración del entorno y lo documenta en el Wiki.
- Al menos el job 'coverage' (SQLite, más simple) pasa exitosamente como baseline.
- El Test Lead aprueba la reanudación tras verificar el estado del entorno.

La reanudación será planificada para minimizar el impacto en el cronograma del Sprint.

6. Actividades y Estimados de Prueba

Este capítulo describe las actividades principales de prueba para el sistema INVENTREE, junto con sus estimaciones de tiempo, diseñadas para validarla. A continuación, se presentan las actividades principales.

ID Act.	Actividad	Responsable	Hrs. Est.	Descripción detallada	Hito
------------	-----------	-------------	--------------	-----------------------	------

INVENTREE Documento de Plan de Pruebas de Integración	UNSA - EPIS Página 45
---	---

A-01	Estudio del pipeline CI/CD real	Test Architect	3 hrs	Leer completamente qc_checks.yaml (691 líneas) y frontend.yaml; identificar todos los jobs, sus dependencias, variables de entorno y comandos exactos	Hito 2
A-02	Análisis de los 452+ métodos de test_api.py	Test Analyst × 2	8 hrs	Leer part, stock, order, build, company, plugin test_api.py; entender qué integra cada clase; mapear fixtures usados	Hito 2
A-03	Fork y configuración del repo en GitHub	Test Architect	1 hr	Fork → habilitar Actions → verificar que qc_checks.yaml se detecta → configurar label 'full-run' para forzar ejecución	Hito 2
A-04	Elaboración del Plan de Integración v2.0	Test Lead + Design	8 hrs	Redactar este documento siguiendo la plantilla del curso; incluir datos reales del código fuente	Hito 2
A-05	Publicación en GitHub Wiki	Test Design	1 hr	Crear página 'Plan-Integracion' en el Wiki del fork; incluir resumen y enlace al documento completo	Hito 2
A-06	Setup del entorno local con Docker (PostgreSQL + Redis)	Test Architect	3 hrs	Instalar Docker; levantar postgres:17 y redis:8; configurar .env con variables del	Hito 3

INVENTREE Documento de Plan de Pruebas de Integración	UNSA - EPIS Página 46
---	---

				qc_checks.yaml; verificar invoke dev.setup-dev	
A-07	Primera ejecución del job 'coverage' (SQLite) en el fork	Test Architect	2 hrs	Push al fork → verificar que job 'coverage' se activa → documentar resultado (PASS/FAIL, tiempo, cobertura)	Hito 3
A-08	Ejecución del job 'postgres' en el fork	Test Architect	2 hrs	Verificar job 'postgres' (PG17+Redis8) en Actions → documentar resultado	Hito 3
A-09	Ejecución del job 'mysql' en el fork	Test Analyst	1 hr	Verificar job 'mysql' en Actions → documentar resultado	Hito 3
A-10	Ejecución del job 'python' (inventree-python) en el fork	Test Analyst	1 hr	Verificar job 'python' en Actions → documentar que el cliente se conecta al servidor real	Hito 3
A-12	Forzar ejecución de migration-tests con label 'full-run'	Test Analyst	2 hrs	Añadir label 'full-run' a una PR en el fork → verificar migration-tests y migrations-checks	Hito 3
A-13	Elaboración del Informe de Resultados	Test Analyst + Design	8 hrs	Completar la tabla de resultados por job; documentar defectos; registrar métricas finales	Hito 3
A-14	Actualización del GitHub Wiki (Informe-Integración)	Test Design	2 hrs	Publicar resultados, screenshots de CI, cobertura obtenida en la página Wiki del fork	Hito 3

A-15	Preparación de la sustentación del Hito 3	Todo el equipo	3 hrs	Demo en vivo del pipeline CI corriendo en el fork; presentación de métricas y análisis de resultados	Hito 3
------	---	----------------	-------	--	--------

7. Personal

Esta sección describe los recursos humanos necesarios para llevar a cabo el esfuerzo de pruebas. Se especifican los roles, responsabilidades, necesidades de contratación y los entrenamientos requeridos.

7.1. Roles, Actividades y Responsabilidades

Para asegurar un proceso de pruebas bien organizado, se definen claramente los roles y responsabilidades del personal involucrado. Se utiliza una matriz RACI para indicar quién es Responsable (R), Aprobador/Responsable final (A), Consultado (C) e Informado (I) en cada actividad clave del proceso de pruebas donde:

- **R – Responsable:** Las personas que ejecutan la tarea o actividad, es decir, son quienes hacen el trabajo.
- **A – Aprobador / Responsable final (Accountable):** La persona que tiene la autoridad final sobre la actividad y se asegura de que se complete correctamente. Solo puede haber un “A” por actividad.
- **C – Consultado:** Personas que deben ser consultadas antes o durante la ejecución de la actividad. Son expertos o partes clave que brindan asesoría o información.
- **I – Informado:** Personas que deben ser notificadas del avance o resultados de la actividad. No participan directamente, pero necesitan estar al tanto.

Actividad	Docente	Test Lead	Test Analyst	Test Architect	Test Design
A-01: Estudio CI/CD pipeline	I	A	C	R	C

INVENTREE Documento de Plan de Pruebas de Integración	UNSA - EPIS Página 48
--	--

A-02: Análisis test_api.py (452+ métodos)	I	A	R	C	R
A-03: Fork y configuración GitHub Actions	I	A	I	R	I
A-04: Plan de Integración v2.0 (este doc)	I	A	C	C	R
A-05: Publicación GitHub Wiki	I	A	I	I	R
A-06: Setup Docker (PG17 + Redis)	I	I	C	R	I
A-07: Ejecución job 'coverage' en el fork	I	A	C	R	I
A-08: Ejecución job 'postgres' en el fork	I	A	R	R	I
A-09: Ejecución job 'mysql' en el fork	I	A	R	C	I
A-10: Ejecución job 'python' en el fork	I	A	R	C	I
A-12: Forzar migration-tests (full-run)	I	A	R	C	I
A-13: Informe de Resultados	I	A	R	C	R
A-14: Actualización Wiki (Informe-Integracion)	I	A	I	I	R
A-15: Sustentación del Hito 3	A	R	C	C	C

7.2. Necesidades de Contratación

En base al análisis de la carga de trabajo estimada y el cronograma definido para las actividades de prueba del sistema TEAMMATES, se ha llegado a la conclusión de que el equipo de pruebas funcionales cuenta con los integrantes necesarios para ejecutar el plan de pruebas de manera efectiva.

7.3. Necesidades de Capacitación

Tema	Audiencia	Duración	Responsable de impartir
Lectura de qc_checks.yaml: cómo funciona paths-filter y los jobs del pipeline	Todo el equipo	1.5 horas	Test Architect (en kick-off)
Estructura de InvenTreeAPITestCase: cómo los test_api.py realizan peticiones HTTP simuladas	Test Analyst, Test Design	2 horas	Test Architect + Lead
Cómo levantar PostgreSQL 17 + Redis 8 con Docker y configurar .env del proyecto	Todo el equipo	1 hora	Test Architect
Cómo interpretar el reporte de Codecov: flags, components, patch vs project	Test Lead, Test Analyst	0.5 horas	Test Architect
GitHub Issues: cómo registrar un defecto de integración con el template del equipo	Todo el equipo	0.5 horas	Test Lead

8. Cronograma

<i>Semana</i>	<i>Fechas</i>	<i>Actividades del Sprint 2</i>	<i>Entregable parcial</i>
<i>Semana 1</i>	<i>02 Jun — 06 Jun 2026</i>	<i>A-01: Estudio completo del pipeline CI/CD; A-02: Análisis de los 452+ métodos de test_api.py por todos los módulos; A-03: Fork del repositorio y habilitación de GitHub Actions</i>	<i>Fork activo con Actions habilitados; anotaciones de análisis en Drive compartido</i>
<i>Semana 2</i>	<i>07 Jun — 10 Jun 2026</i>	<i>A-04: Elaboración del Plan de Integración v2.0 (este documento); A-05: Publicación en GitHub Wiki; revisión final del documento</i>	<i>PLAN DE PRUEBAS DE INTEGRACIÓN v2.0 ← ENTREGABLE HITO 2 — 10 Jun 2026</i>
<i>Semana 3</i>	<i>11 Jun — 17 Jun 2026</i>	<i>[Hito 3] A-06: Setup Docker; A-07: Job coverage; A-08: Job postgres; A-09: Job mysql; A-10: Job python; primera ejecución completa del pipeline</i>	<i>Capturas de GitHub Actions; métricas de cobertura del primer run</i>
<i>Semana 4</i>	<i>18 Jun — 24 Jun 2026</i>	<i>[Hito 3] A-11: Playwright tests; A-12: migration-tests forzados; A-13: Informe de Resultados; A-14: Wiki actualizado [Playwright excluido del plan de integración — prueba de Aceptación]</i>	<i>Informe de Resultados de Pruebas de Integración completo; Wiki con evidencias</i>
<i>Sustentación</i>	<i>Según calendario</i>	<i>[Hito 3] A-15: Presentación al docente — demo del CI en vivo en el fork, análisis de cobertura, defectos encontrados</i>	<i>Sustentación Hito 3 con evidencias en GitHub</i>